

Current at 39

After Next () :

Current at 10.

After Insert (12) :

Head $\rightarrow$ [39] $\rightarrow$ [2] $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [15] $\rightarrow$ NULL

Current at 12.

2)

Array

66 111 212 323 432 543 643 777 888

Case 1 : Key = 111

- Low = 0 High = 9 Mid = 4 $\rightarrow$ 432
- $111 < 432 \rightarrow$ High = 3
- Mid = 1 $\rightarrow$ 111 found

Position = 2 (Index = 1).

Case 2 : Key = 888

- Mid = 4 $\rightarrow$ 432
- Mid = 7 $\rightarrow$ 643
- Mid = 8 $\rightarrow$ 777
- Mid = 9 $\rightarrow$ 888 found

Position = 10 (Index = 9)

Case 3 : Key = 999

- Mid = 4 $\rightarrow$ 432
- Mid = 7 $\rightarrow$ 643
- Mid = 8 $\rightarrow$ 777

• Mid = 9 $\rightarrow$ 888

Low > High

Result : Element not found.

Time complexity

• Best case : $O(1)$

• Avg case : $O(\log n)$

• Worst case : $O(\log n)$.

3)

Stack size = 5

Operations :

Push (45)

Push (22)

Push (11)

Push (77)

Pop()

Push (44)

Push (33)

Pop()

Pop()

Pop()

Execution :

[]

Push 45 $\rightarrow$ [45]

Push 22 $\rightarrow$ [45, 22]

Push 11 $\rightarrow$ [45, 22, 11]

Push 77 $\rightarrow$ [45, 22, 11, 77]

Pop $\rightarrow$ [45, 22, 11]

Push 44 $\rightarrow$ [45, 22, 11, 44]

Push 33 $\rightarrow$ [45, 22, 11, 44, 33]

Pop $\rightarrow$ [45, 22, 11, 44]

Pop $\rightarrow$ [45, 22, 11]

Pop $\rightarrow$ [45, 22]

final stack:

top

↓

[22]

[45]

Top (Peak) Element = 22

Top Index = 1.

4)

Expression:

$(a+b) + c * (d+e) + f * (g+h)$

Prefix:

$* + + + ab * c + de f + gh.$

or without spaces:

$* + + + ab * c + def + gh$

Post fix:

$ab + cdef * + f + gh + *$

5)

Simple Queue

Follows FIFO.

Fair to all customers.

Waiting time is equal.

Normal customers wait less predictable.

Enqueue : $O(1)$

Dequeue : $O(1)$

ANALYSIS:

- Simple Queue : Ensures fairness because customers are served in the order they arrive.
- Priority Queue : Improve service for VIP customers but reduces fairness for regular customers.
- Waiting Time : VIP waiting time decreases, while regular customers waiting time may increase.
- Complexity : Priority queues require additional processing, increasing computational complexity compared to a simple queue.

Priority Queue

Highest priority served first.

VIP customers get preference.

VIP waiting time decreases.

Normal customers may wait longer.

Enqueue : $O(\log n)$

Dequeue : $O(\log n)$